Q-Learning GridWorld Agent – Implementation & Testing Report

1. Introduction

This project implements a reinforcement learning agent based on the Q-learning algorithm, designed to operate in a GridWorld environment accessed through an external API. The agent learns by interacting with the environment, receiving rewards, and updating its policy over time. Unlike standard reinforcement learning simulations, this system works in a real API-driven environment, which introduces challenges such as inconsistent responses, missing state information, and network delays. Because of this, the system is designed with a strong emphasis on robustness, fault tolerance, and persistence.

2. System Architecture Overview

The system is divided into several interconnected modules, each responsible for a specific part of the reinforcement learning pipeline. This modular structure improves maintainability and makes debugging easier. At the core of the system is the **grid representation (GridSpec),** which defines the environment size and provides helper functions for converting between 2D coordinates and 1D state indices. This conversion is essential because the Q-table is indexed using a single-dimensional state space. The function *to_state(row, col)* performs this conversion, while *contains_state(state)* ensures that any state used by the agent is valid within the grid boundaries.

In addition to the learning agent, the system includes a *CampaignRunner* module that manages multiple worlds and long-term progression. It tracks goal discovery, counts successful goal hits, and determines when to switch between worlds. Depending on the state of the environment, the system dynamically selects between BFS-based shortest path execution, coordinate-based exploitation, or standard Q-learning.

3. Configuration System (*QConfig*)

The *QConfig* class is responsible for storing all hyperparameters and runtime settings used by the agent. This includes learning parameters such as the learning rate (*alpha*) and discount factor (*gamma*), as well as exploration settings like epsilon and its decay rate. In addition to learning parameters, this configuration also controls operational behavior. For example, *move_delay_sec* and *enter_delay_sec* enforce API rate limits, while *max_episodes* and *max_steps_per_episode* define the training duration. The purpose of this class is to centralize all tunable parameters so that experiments can be adjusted without modifying the core logic of the program.

4. API Communication Layer (*GridWorldClient*)

The *GridWorldClient* class is responsible for all communication with the external GridWorld API. It acts as a bridge between the agent and the environment. The function *enter_world(world_id)* is used to place the agent into a specific environment. The function *move(world_id, move)* sends an action (such as North, South, East, or West) to the environment and returns the resulting state and reward. The function *get_location()* retrieves the current

position of the agent, which is especially useful when the API does not explicitly return the next state. This class also includes a helper function *_check_response(data)* that validates API responses. If the API returns an error code instead of a successful response, this function raises an exception, ensuring that invalid data does not propagate into the learning system.

5. Response Parsing System (*GridWorldResponseParser*)

The *GridWorldResponseParser* plays a critical role in interpreting API responses, which are often inconsistent in format.

The function *parse_location_state(data)* extracts the agent's current state from different possible response formats. It can handle direct integer states, string-based states, and coordinate formats like "*row:col*". This makes the system flexible enough to handle variations in API output. Another important function is *parse_move_result(data, current_state),* which interprets the result of an action. It extracts the reward, determines the next state, and checks whether the episode has ended. It also detects special conditions such as blocked movements or invalid actions by analyzing message content.

If the API does not return a valid next state, this function works together with the agent to either recover the state or assume a self-loop transition. To handle unreliable API responses, the system uses *_safe_get_location()* which retries multiple times before assuming failure. This ensures robustness against missing or inconsistent state data.

6. Q-Learning Implementation (*QLearner*)

The *QLearner* class contains the core reinforcement learning logic.

The function *choose_action(q_table, state, epsilon)* implements the ε-greedy policy. With probability ε, the agent selects a random action to encourage exploration. Otherwise, it selects the action with the highest Q-value. If multiple actions share the same value, one is chosen randomly to avoid bias.

The function *update_q(q_table, state, action, reward, next_state, done)* updates the Q-table using the standard Q-learning update rule. It calculates the temporal difference error between the current Q-value and the expected future reward, then adjusts the Q-value accordingly. If the episode has ended, the future reward is ignored.

7. Agent Controller (*QLearningAgent*)

The *QLearningAgent* class is the central component that coordinates all system modules.

The function *enter_world_once()* attempts to place the agent into the environment. If the API indicates that the agent is already inside a world, this function retrieves the current state instead of re-entering. This prevents unnecessary resets and ensures continuity.

The function *get_current_state()* retrieves the agent's current position using the API and ensures that it is valid before returning it. The function *run_episode(initial_state)* is the main training

loop for a single episode. It repeatedly selects actions, sends them to the environment, processes responses, and updates the Q-table. It also tracks important statistics such as total reward, number of blocked moves, and invalid actions.

The function *_safe_get_location()* is used as a fallback mechanism when the API does not return a valid next state. It retries multiple times before failing and ensures that training can continue even under unreliable network conditions.

The agent also includes advanced exploration strategies beyond standard ε-greedy selection. Functions such as *_choose_exploration_action()* and *_frontier_action()* help prioritize unexplored or promising states. The agent maintains a memory of safe and unsafe transitions using *_remember_safe_edge()* and *_remember_bad_transition()*, which improves long-term decision-making. Additionally, reward shaping techniques such as novelty bonuses and loop penalties are applied to encourage efficient exploration and prevent repetitive behavior.

The system also uses reward shaping to improve learning efficiency. This includes loop penalties for repeated states, novelty-based rewards for visiting new states, and terminal-based reward adjustments for goal and hazard states. These shaping mechanisms are applied inside *run_episode()* and *_state_novelty_reward().*

Finally, the function *train()* manages the full training process across multiple episodes. It calls *run_episode()* repeatedly, handles saving of progress, and updates exploration parameters over time.

8. Testing Strategy (*testmain.py*)

The testing suite verifies correctness of all major components.

Parser-related tests ensure that coordinate conversion works correctly, blocked movements are properly handled, and terminal states are detected accurately.

Learner tests validate that action selection behaves correctly, especially in cases where multiple actions have identical Q-values. This ensures that the agent maintains proper randomness and does not become biased.

Agent-level tests focus on robustness. They confirm that the system can restore its state from saved metadata, handle missing API responses gracefully, and correctly retry failed location requests. Additional tests ensure that API rate limits are respected.

Additional tests covered campaign-level persistence and recovery mechanisms. Functions such as *CampaignStore.load()* and *CampaignStore.save()* were tested to ensure that progress tracking remains consistent across sessions. The system was also tested for robustness in handling missing API responses using *_safe_get_location()*. Path execution logic, including *find_shortest_known_goal_path()* and *follow_planned_goal_path()*, was validated to ensure correct execution of planned optimal routes.

9. System Strengths

The system is designed with strong fault tolerance in mind. Each function is built to handle unexpected or inconsistent inputs without crashing the entire system. The separation between API handling, parsing, learning, and storage ensures that each part can be tested and improved independently.

Another strength is the persistence mechanism, which ensures that learning progress is saved and can be resumed even after interruptions. This is particularly important in long training sessions.

10.Conclusion

This project successfully implements a Q-learning agent capable of operating in a real-world API-based GridWorld environment. The system is highly modular, with clearly defined responsibilities across different components.

Key functions such as *enter_world_once*, *parse_move_result*, *choose_action*, and *update_q* form the backbone of the learning process. Together, they enable the agent to interact with an uncertain environment, learn from feedback, and gradually improve its performance.

The system extends beyond traditional Q-learning by integrating planning-based techniques such as BFS shortest path search and coordinate-based exploitation. This hybrid approach allows the agent to combine reinforcement learning with explicit path planning, resulting in faster and more reliable goal completion.

The unit tests confirm that all major components behave correctly under both normal and edge case scenarios. Overall, the system demonstrates a practical and robust application of reinforcement learning beyond controlled simulations.